value unless explicitly allowed.

Any type in TypeScript can be used as the basis of an array with two different syntaxes that work identically: `string[]` or `Array<string>`. The second syntax is an example of a more general TypeScript feature called *generic types*, which allows the same type structure—in this case an `Array`—to have a different internal type—in this case a `string`—while still retaining the same behavior no matter the internal type.

Generic types allow you to have type checking in cases where the fact that the type is consistent across a class or function is more important than what specific type is involved.

Data structures are a common use case for generic types. If you have a type that is a list, for example, and you want to write a method that returns the first element of the list, without generics, you might have to write that function signature differently for each potential type of data you might have in the list:

```
function getFirst(list: AStringList): string
function getFirst(list: ANumberList): number
function getFirst(list: AUserList): User
```

and so on. But there is a pattern here: the return value is always the same type as the values that make up the list.

TypeScript allows us to use generics to represent the pattern, like this:

```
function getFirst(list: AList<T>): T
```

The angle brackets here represent the generic type and the `T` is just an identifier and could be anything (or at least, anything starting with a capital letter). Single-letter identifiers are usually used, at least in part, to make a clear distinction between generics and specific types.

The elements of a TypeScript array need to all be of the same type. If for some reason you need a linear structure that has multiple types, first you should think really hard about whether that is what you really want (most likely you want a